

Simulating Sensors in PT

Frank Walsh

Context

- You want to create a project but you don't have all the devices required
- You want to test scenarios that are difficult to create/repeat in real world
 - High/low temperatures and extreme environmental conditions
 - Repeatable Tests - Test your software/hardware use cases after a change is made
- Prototyping/feasibility: test a solution before you purchase/acquire hardware

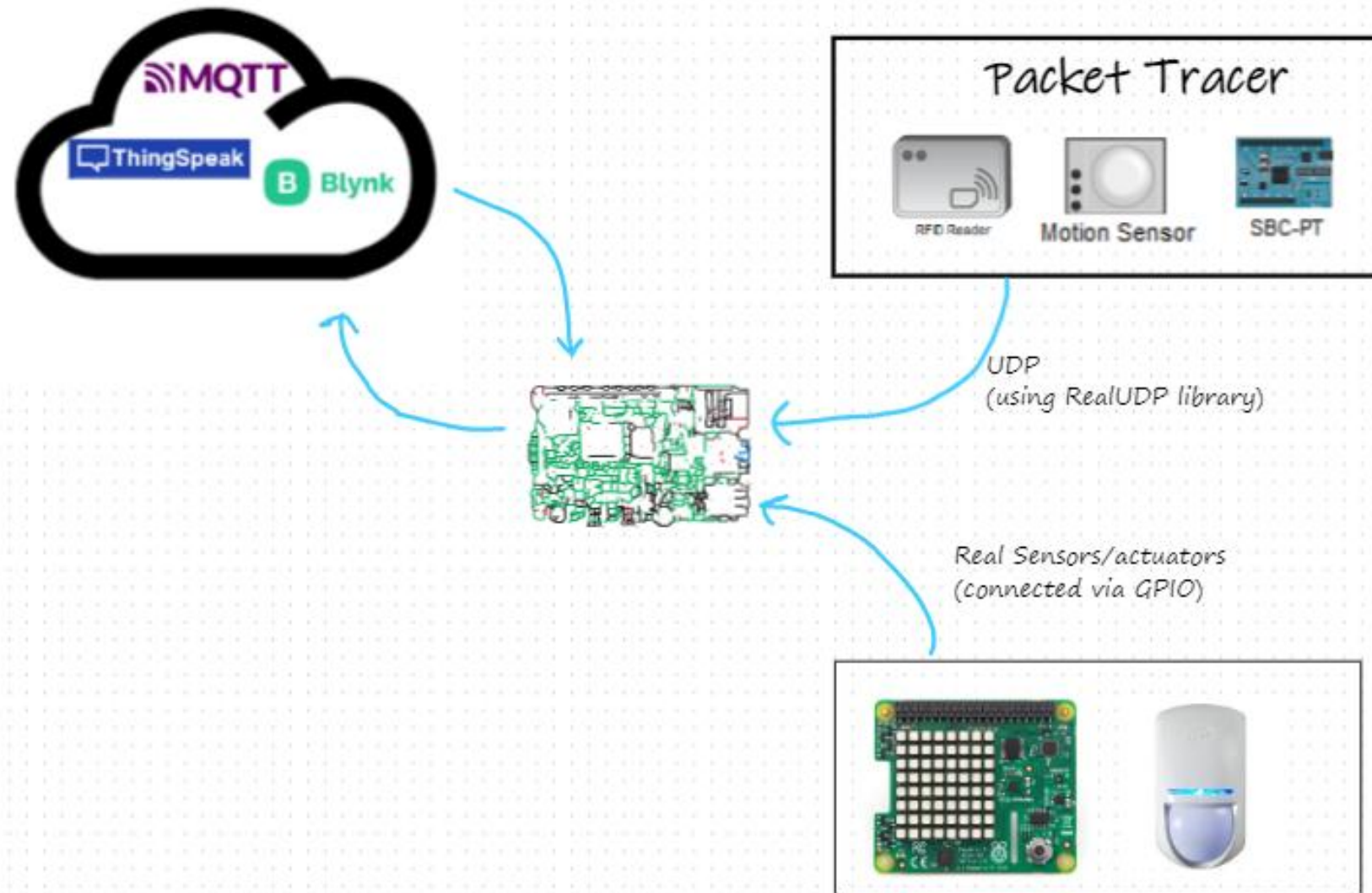


Asside: Digital Twin

A digital twin is a digital representation of a physical asset (e.g., a sensor, device, or system) that models its behaviour and state, often using live and historical data from the real asset to stay synchronized and support simulation and analysis.

- **Near Real-Time Operation:** reflects the state of a corresponding physical counterpart or simulates it based on defined parameter
- **Repeatable Testing:** Consistent, repeatable testing scenarios, ensuring that edge cases, errors, and performance can be tested
- **Integration Testing:** integrating and testing systems, such as IoT applications, without requiring the actual physical devices to be present in early-stage development.
- **Scalability and Flexibility:** can simulate multiple devices and can be easily modified to adjust parameters or simulate different environmental conditions and various use cases.

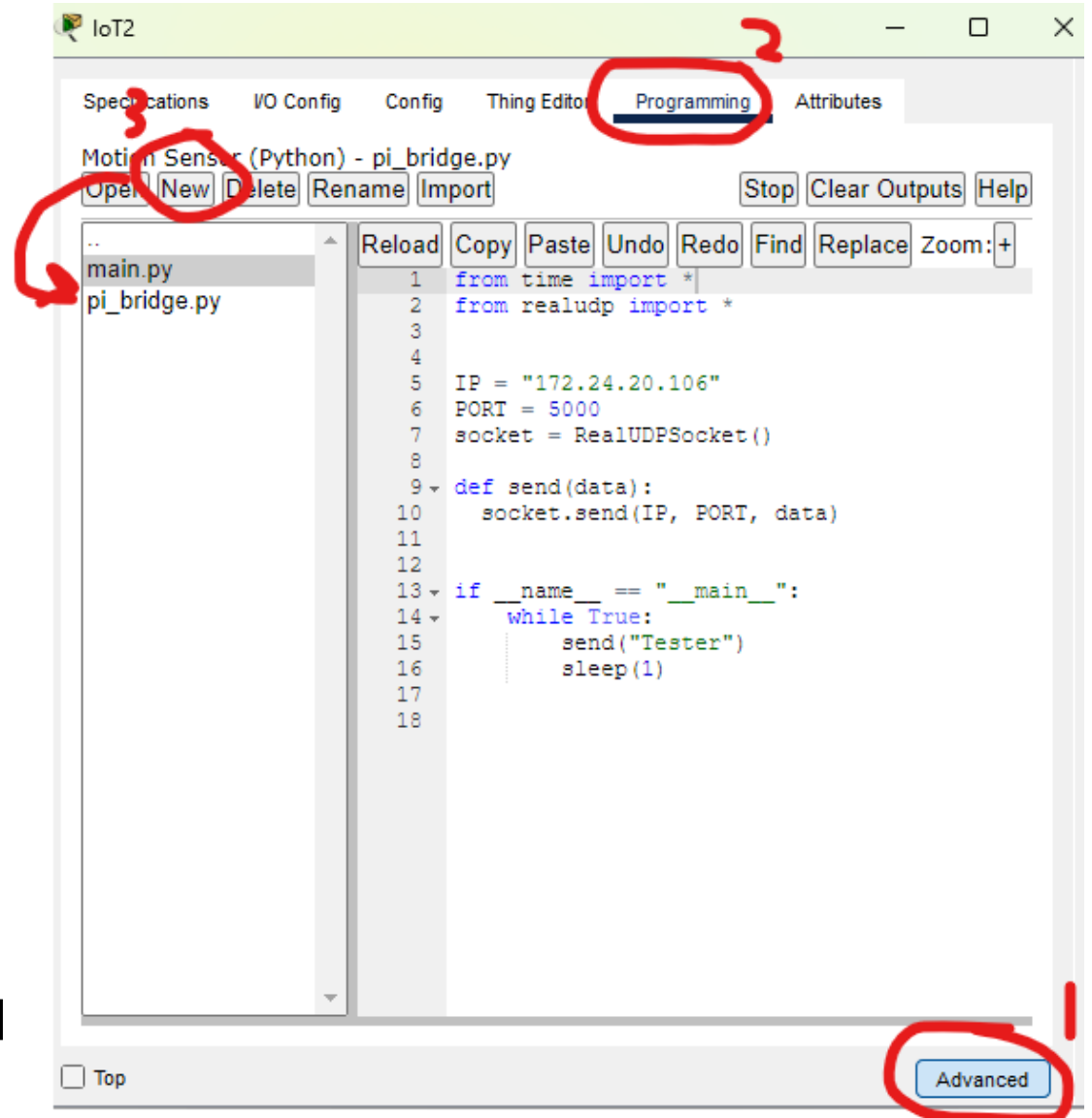
Generalised Design:



1. Update Sensors

Update PT sensors to send data via UDP

- Add sensor to PT workspace.
- Open Sensor configuration and click on Advanced
- Select “Programming” tab
- Select Python version of running program
- Add “bridging” module
 - Sends UDP datagram with sensor data to external UDP server
- Import bridging module and call to send data.



2. Sensor Listener Module

Develop a sensor module that “listens” for PT sensor data and can provide it to other programs

- Develop sensor_listener module
 - Instances receive data from sensors (or other devices) over a network using UDP
 - Import into other programs to receive sensor data.
 - Uses Threading to start the UDP listening process separate to the main program
 - Deploy with main program

```
import socket
import threading

class SensorListener:
    def __init__(self, host='0.0.0.0', port=5000, buffer_size=1024):
        """
        Initialise the UDP Listener.

        :param host: IP address to bind the UDP listener (default is 0.0.0.0 for all interfaces)
        :param port: Port to listen on
        :param buffer_size: Size of the data buffer for receiving UDP packets
        """
        self.host = host
        self.port = port
        self.buffer_size = buffer_size
        self.running = False
        self.callback = None

    def start(self):
        """
        Start in a separate thread.
        """
        self.running = True
        threading.Thread(target=self._listen, daemon=True).start()
```

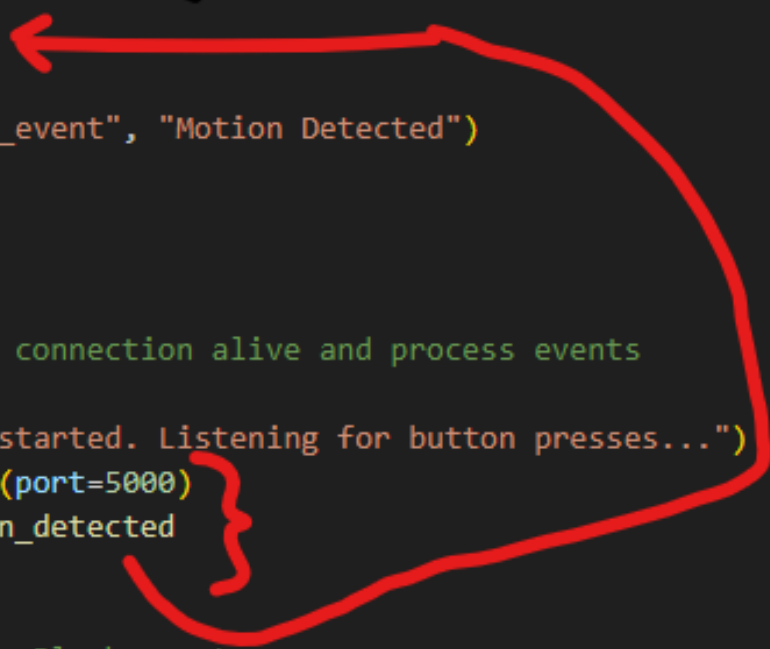
3. Import into Main Program

- Import Sensor Listener into main program
 - Instead/substituting sensor
- Create instance and assign “callback” function to handle sensor data events as they occur.

```
# Initialise the Blynk instance
blynk = BlynkLib.Blynk("RVIj9R2_Q+XXXXXXXXXXXXiKeAIoKmplF")

def motion_detected(data):
    print("motion detected")
    blynk.log_event("movement_event", "Motion Detected")
    blynk.virtual_write(0, 1)
    sleep(2)
    blynk.virtual_write(0, 0)

# Main loop to keep the Blynk connection alive and process events
if __name__ == "__main__":
    print("Blynk application started. Listening for button presses...")
    listener = SensorListener(port=5000)
    listener.callback = motion_detected
    listener.start()
    while True:
        blynk.run() # Process Blynk events
```



DEMO!

- PT Simulation and Code available on Website...